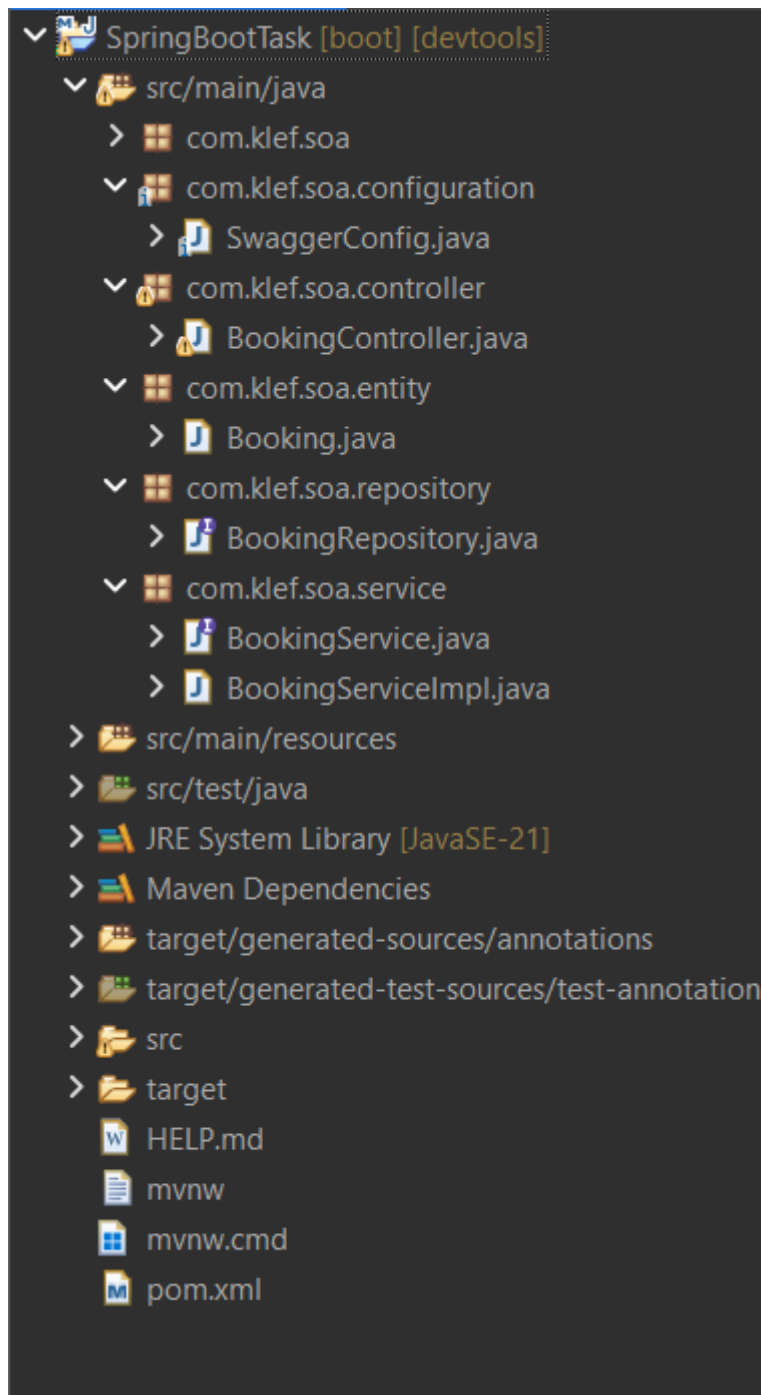
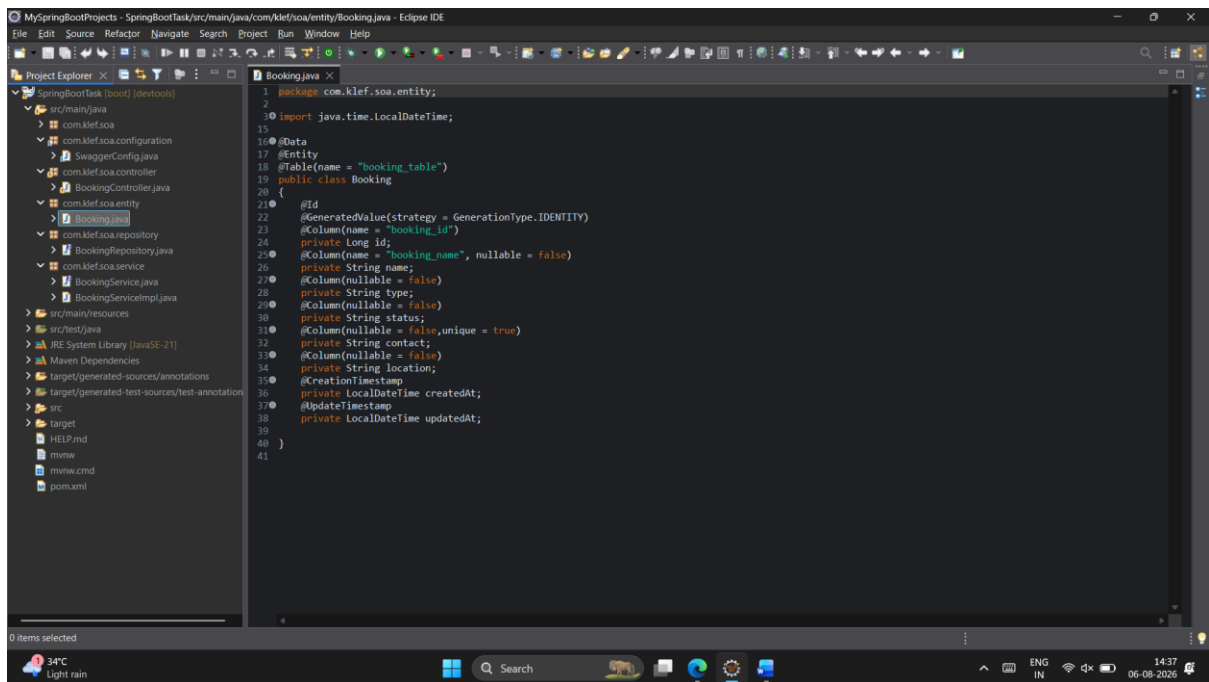


Entity: Booking

Project Structure:



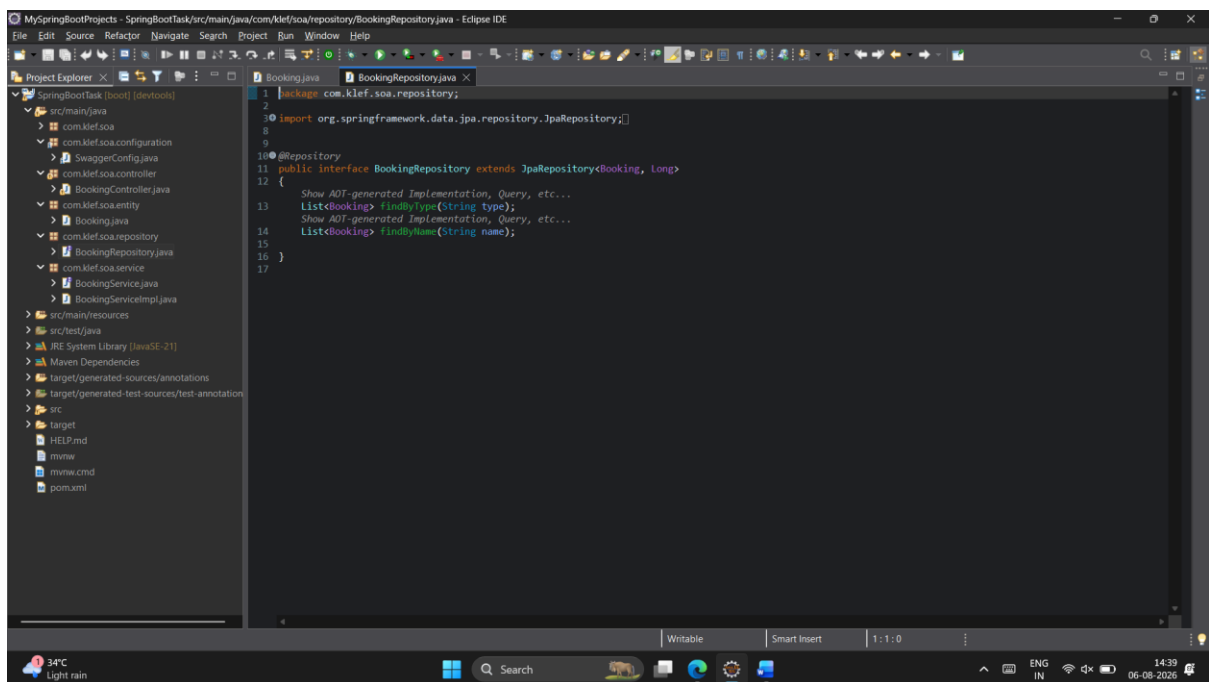
Booking.java:



The screenshot shows the Eclipse IDE with the 'Booking.java' file open. The Project Explorer on the left shows the project structure, including 'com.kief.soa.entity'. The main editor displays the following Java code:

```
1 package com.kief.soa.entity;
2
3 import java.time.LocalDateTime;
4
5
6 @Data
7 @Entity
8 @Table(name = "booking_table")
9 public class Booking
10 {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     @Column(name = "booking_id")
14     private Long id;
15     @Column(name = "booking_name", nullable = false)
16     private String name;
17     @Column(nullable = false)
18     private String type;
19     @Column(nullable = false)
20     private String status;
21     @Column(nullable = false, unique = true)
22     private String contact;
23     @Column(nullable = false)
24     private String location;
25     @CreationTimestamp
26     private LocalDateTime createdAt;
27     @UpdateTimestamp
28     private LocalDateTime updatedAt;
29 }
30
31
32
33
34
35
36
37
38
39
40
41
```

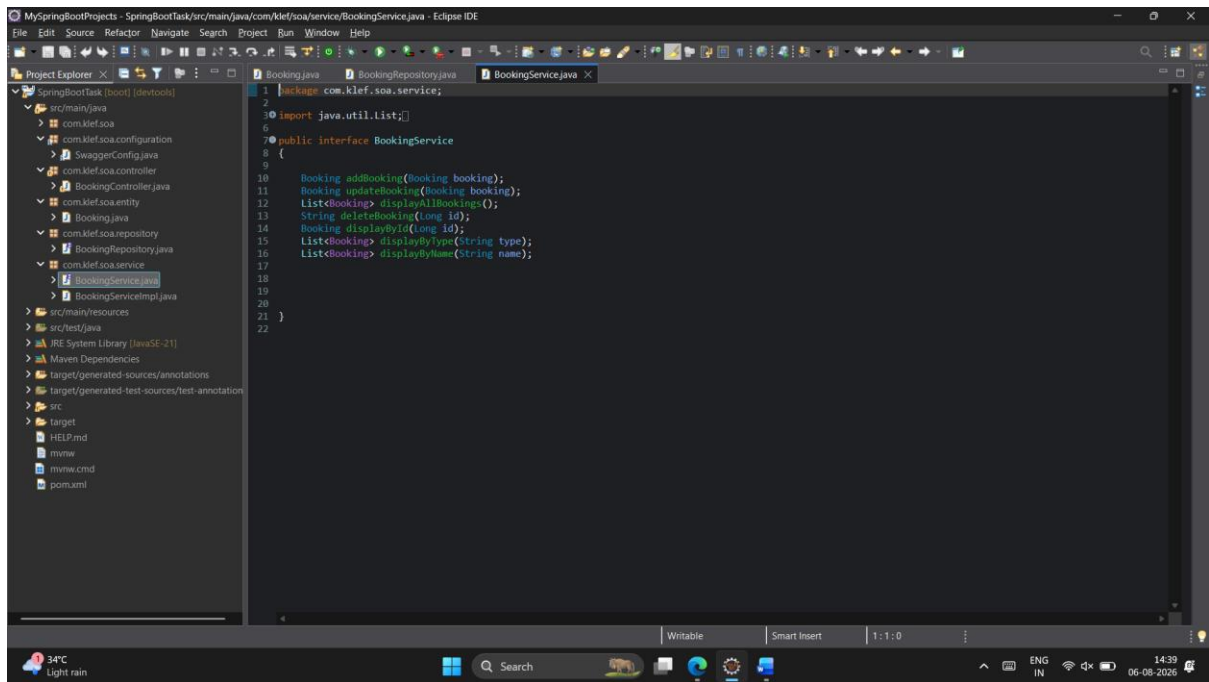
BookingRepository.java:



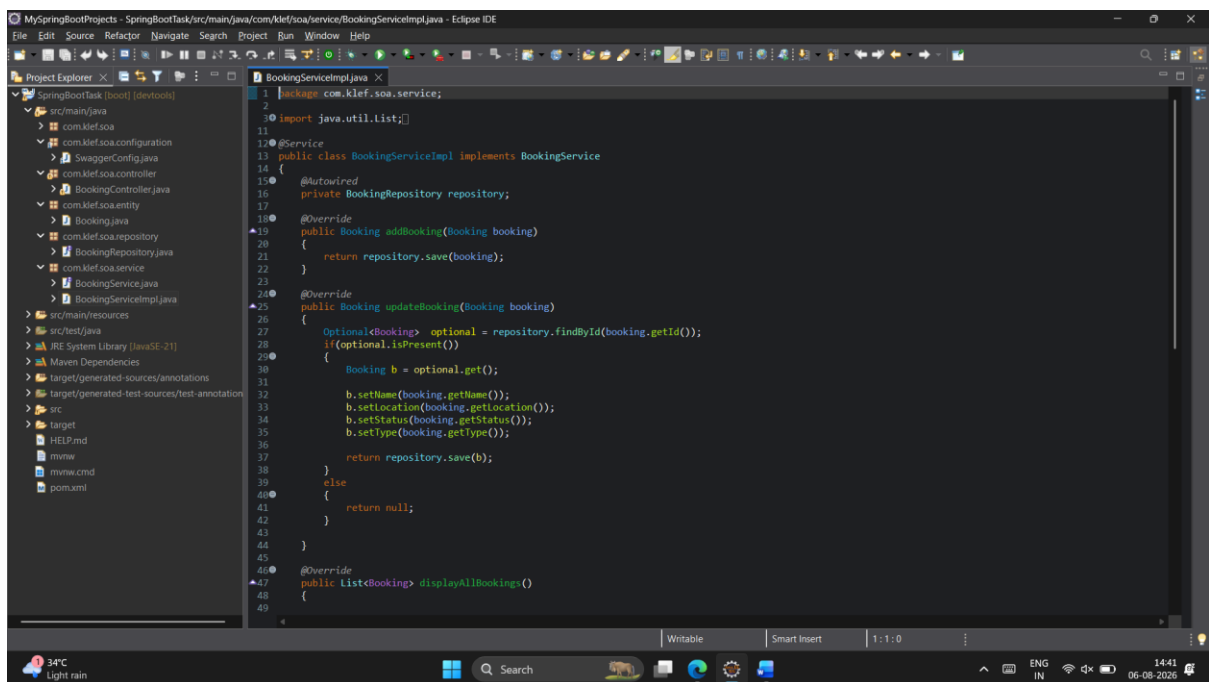
The screenshot shows the Eclipse IDE with the 'BookingRepository.java' file open. The Project Explorer on the left shows the project structure, including 'com.kief.soa.repository'. The main editor displays the following Java code:

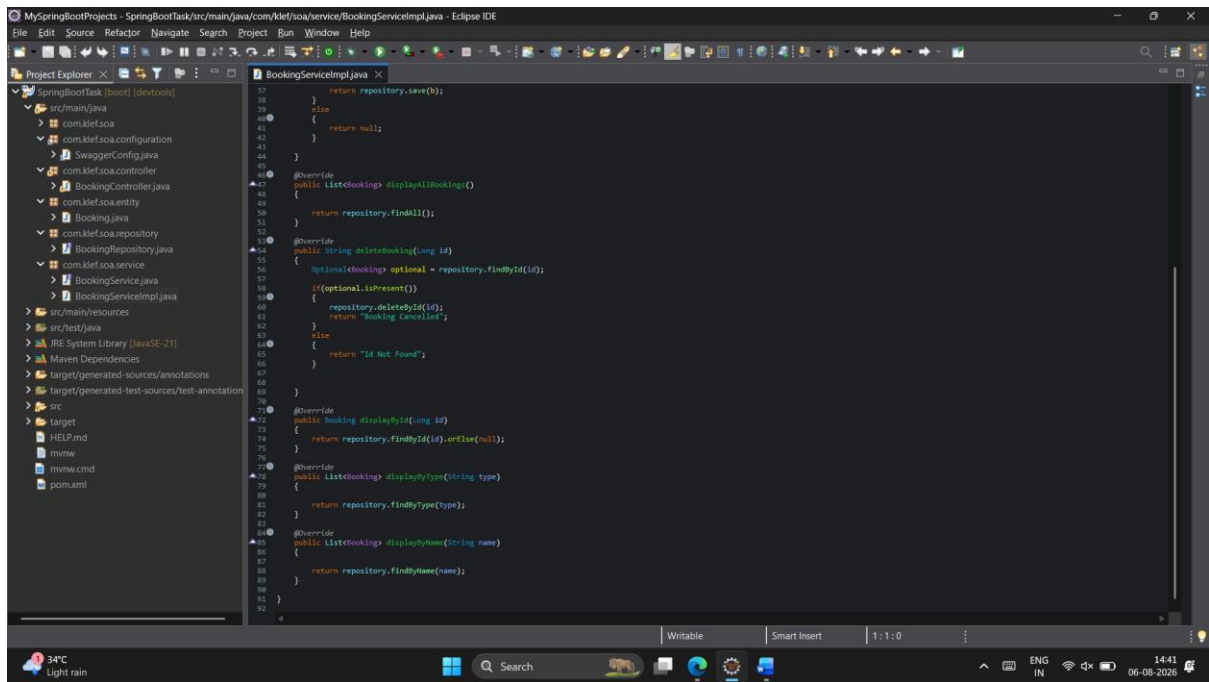
```
1 package com.kief.soa.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5
6 @Repository
7 public interface BookingRepository extends JpaRepository<Booking, Long>
8 {
9     // Show AOT-generated Implementation, Query, etc...
10     List<Booking> findByName(String type);
11     // Show AOT-generated Implementation, Query, etc...
12     List<Booking> findByName(String name);
13 }
14
15
16
17
```

BookingService.java:



BookingServiceImpl.java:

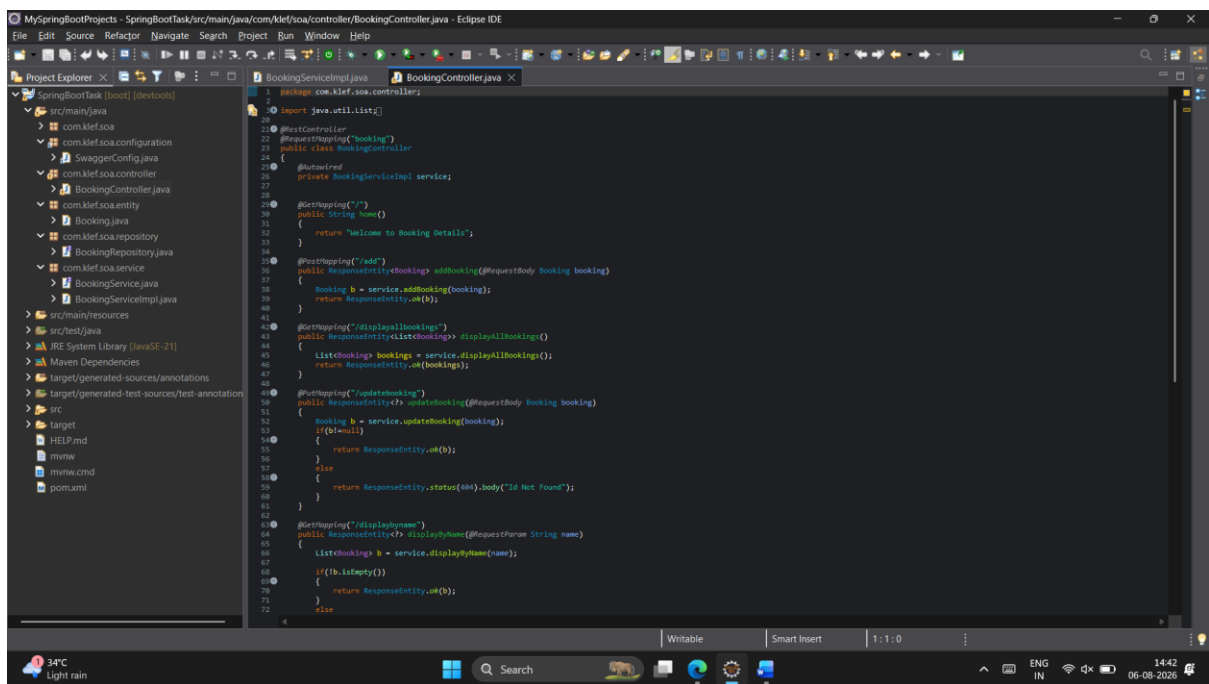




The screenshot shows the Eclipse IDE with the 'BookingServiceImpl.java' file open. The Project Explorer on the left shows the project structure for 'SpringBootTask'. The main editor displays the following code:

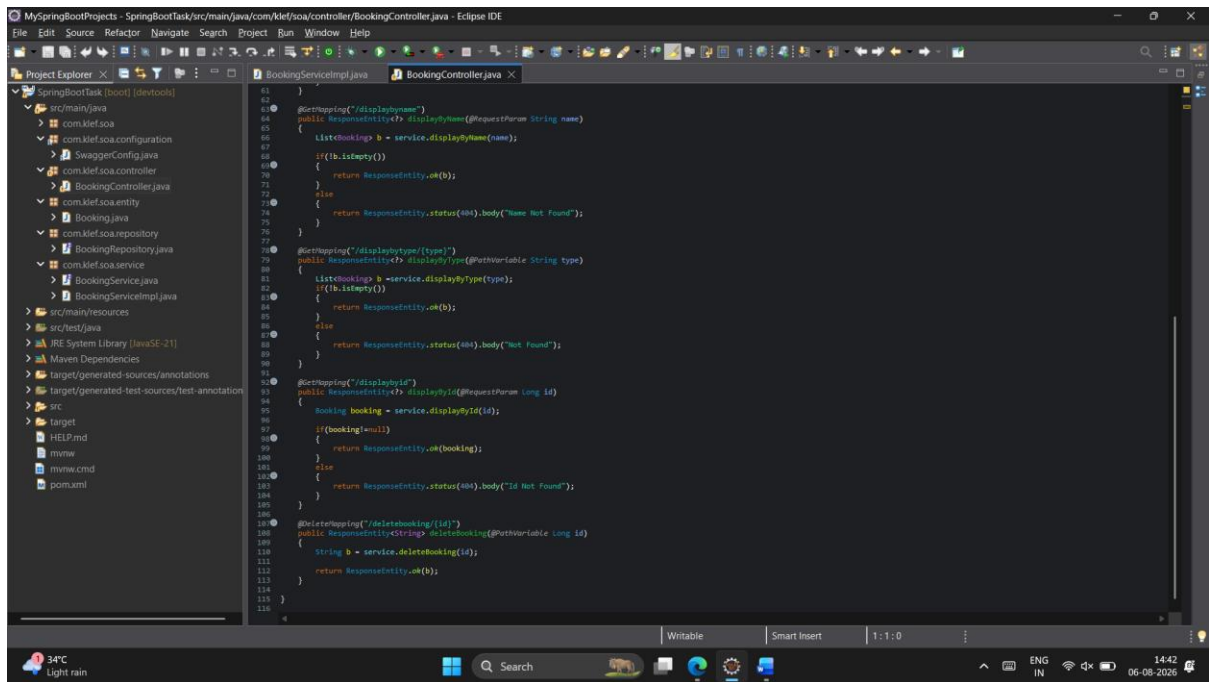
```
17     return repository.save(b);
18     }
19     else
20     {
21         return null;
22     }
23     }
24
25     @Override
26     public List<Booking> displayAllBookings()
27     {
28         return repository.findAll();
29     }
30
31     @Override
32     public String deleteBooking(Long id)
33     {
34         Optional<Booking> optional = repository.findById(id);
35         if(optional.isPresent())
36         {
37             repository.deleteById(id);
38             return "Booking Cancelled";
39         }
40         else
41         {
42             return "Id Not Found";
43         }
44     }
45
46     @Override
47     public Booking displayById(Long id)
48     {
49         return repository.findById(id).orElse(null);
50     }
51
52     @Override
53     public List<Booking> displayByType(String type)
54     {
55         return repository.findByType(type);
56     }
57
58     @Override
59     public List<Booking> displayByName(String name)
60     {
61         return repository.findByName(name);
62     }
63 }
```

BookingController.java:

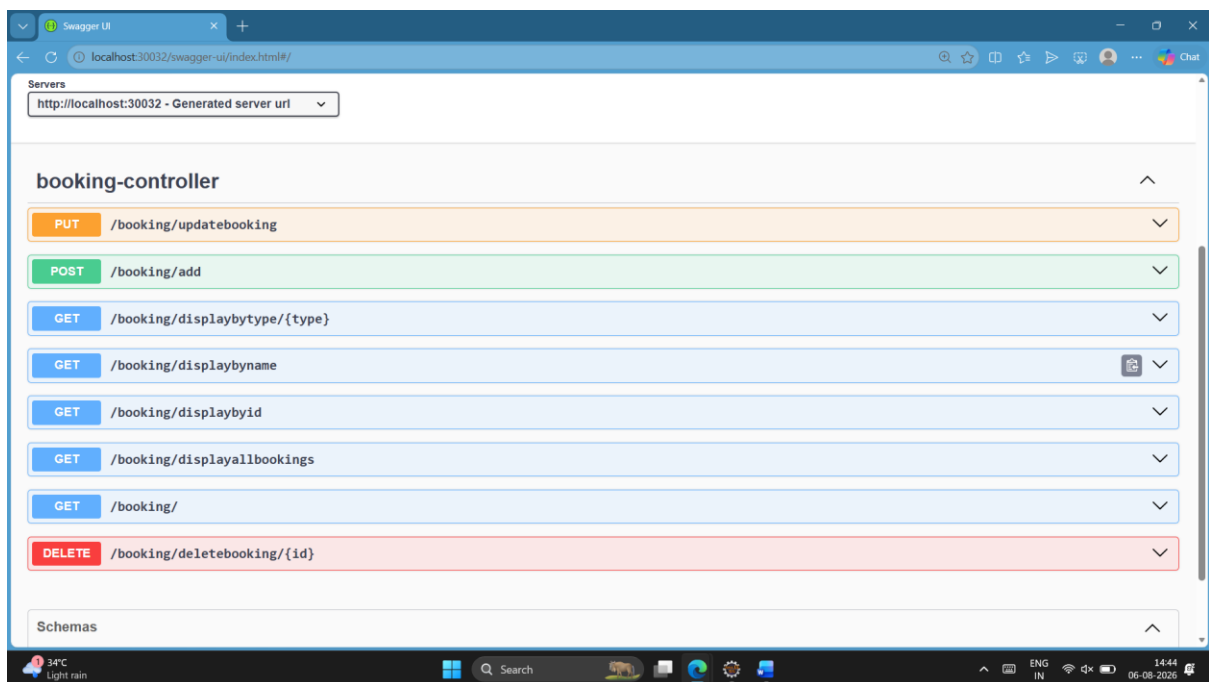


The screenshot shows the Eclipse IDE with the 'BookingController.java' file open. The Project Explorer on the left shows the project structure for 'SpringBootTask'. The main editor displays the following code:

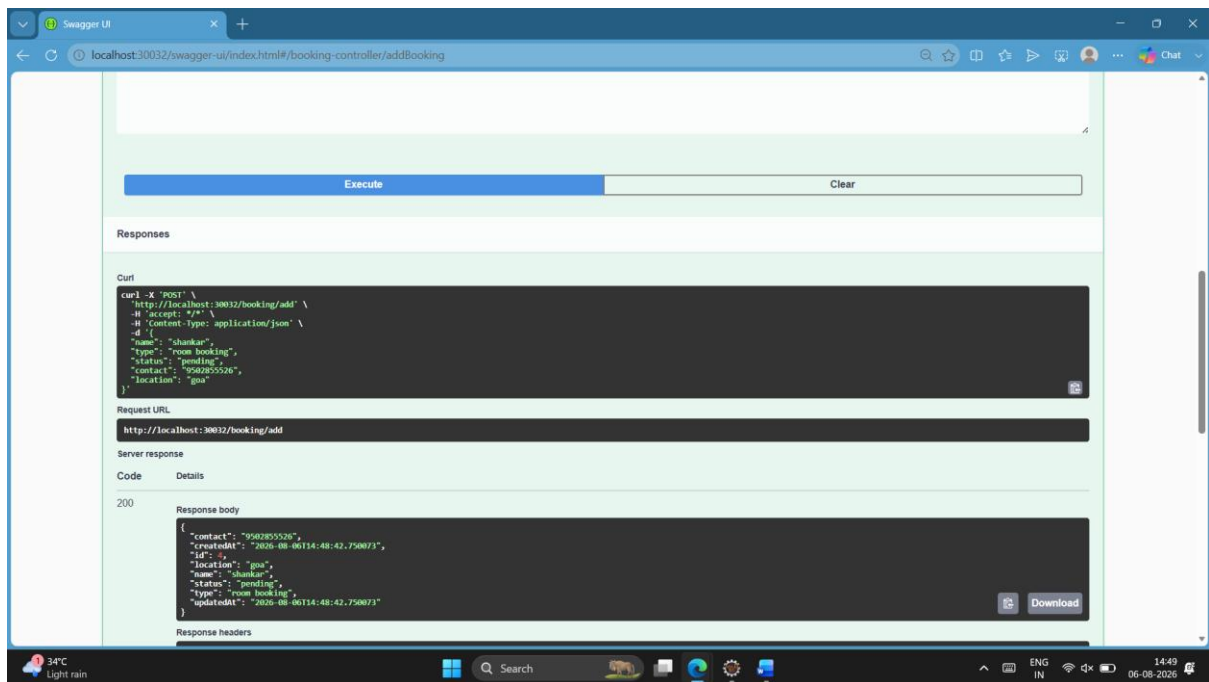
```
1 package com.klef.soa.controller;
2
3 import java.util.List;
4
5 @RestController
6 @RequestMapping("booking")
7 public class BookingController
8 {
9     @Autowired
10     private BookingServiceImpl service;
11
12     @GetMapping("/")
13     public String home()
14     {
15         return "Welcome to Booking Details";
16     }
17
18     @PostMapping("/add")
19     public ResponseEntity<Booking> addBooking(@RequestBody Booking booking)
20     {
21         Booking b = service.addBooking(booking);
22         return ResponseEntity.ok(b);
23     }
24
25     @GetMapping("/displayallbookings")
26     public ResponseEntity<List<Booking>> displayAllBookings()
27     {
28         List<Booking> bookings = service.displayAllBookings();
29         return ResponseEntity.ok(bookings);
30     }
31
32     @PostMapping("/updatebooking")
33     public ResponseEntity<?> updateBooking(@RequestBody Booking booking)
34     {
35         Booking b = service.updateBooking(booking);
36         if(b!=null)
37         {
38             return ResponseEntity.ok(b);
39         }
40         else
41         {
42             return ResponseEntity.status(404).body("Id Not Found");
43         }
44     }
45
46     @GetMapping("/displaybyname")
47     public ResponseEntity<?> displayByName(@RequestParam String name)
48     {
49         List<Booking> b = service.displayByName(name);
50         if(b.isEmpty())
51         {
52             return ResponseEntity.ok(b);
53         }
54         else
55         {
56             return ResponseEntity.ok(b);
57         }
58     }
59 }
```



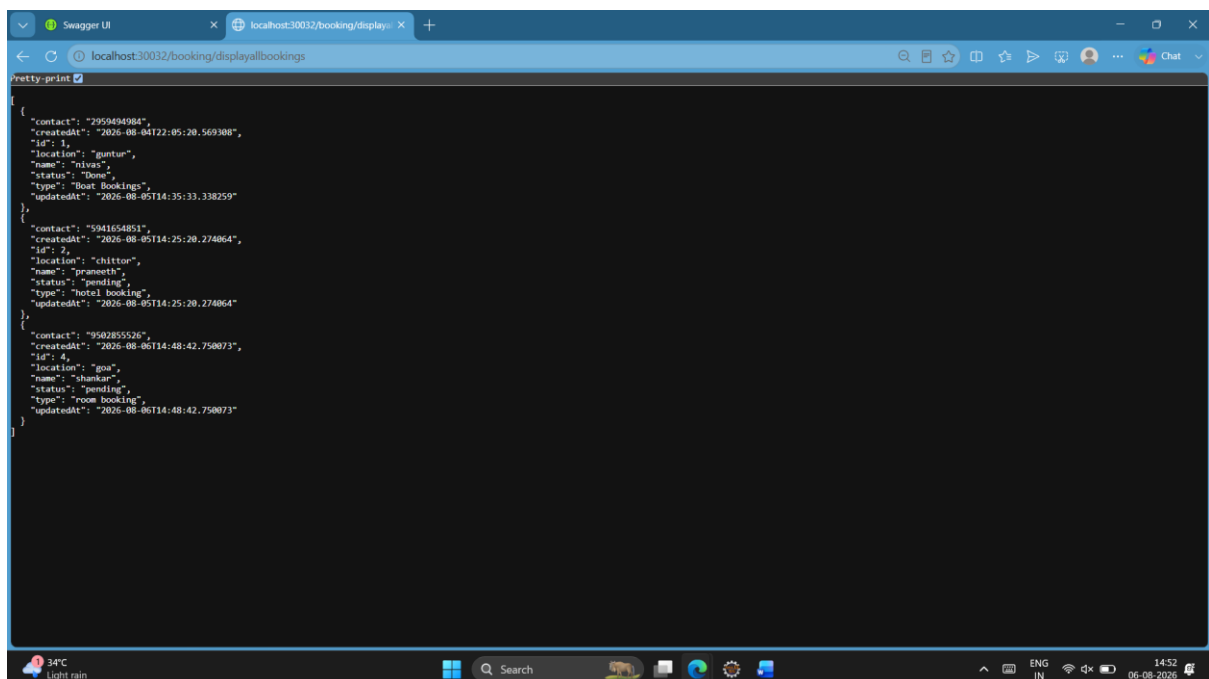
Swagger:



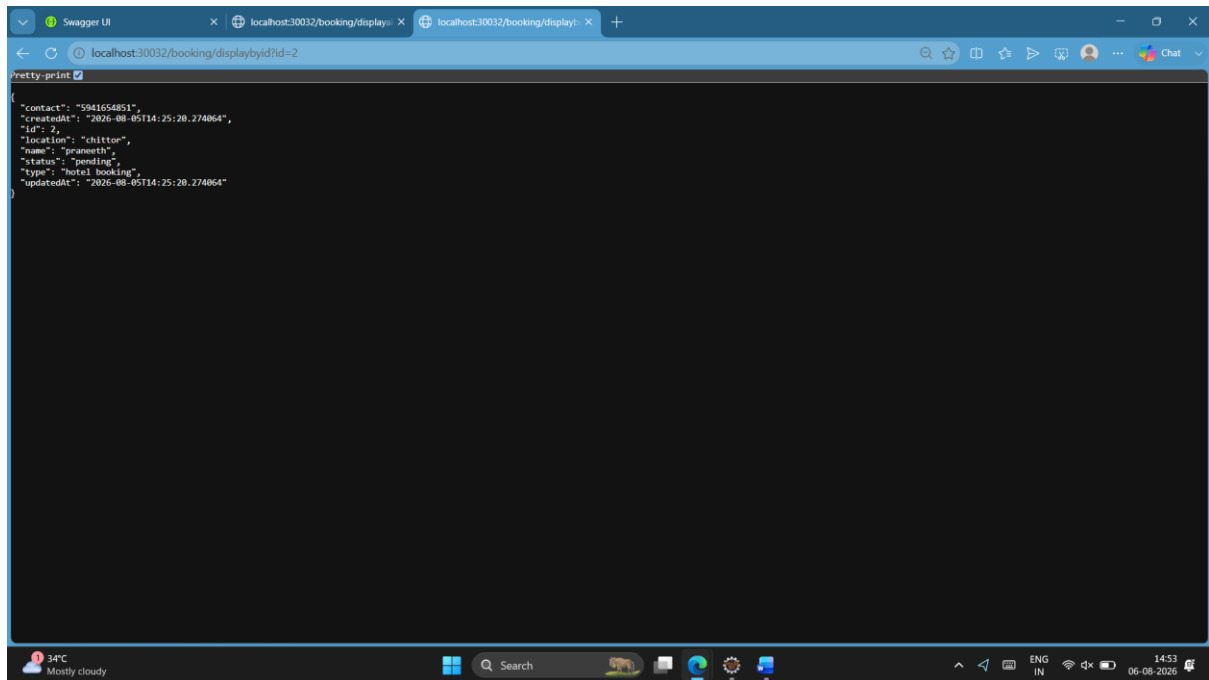
<http://localhost:30032/booking/add>



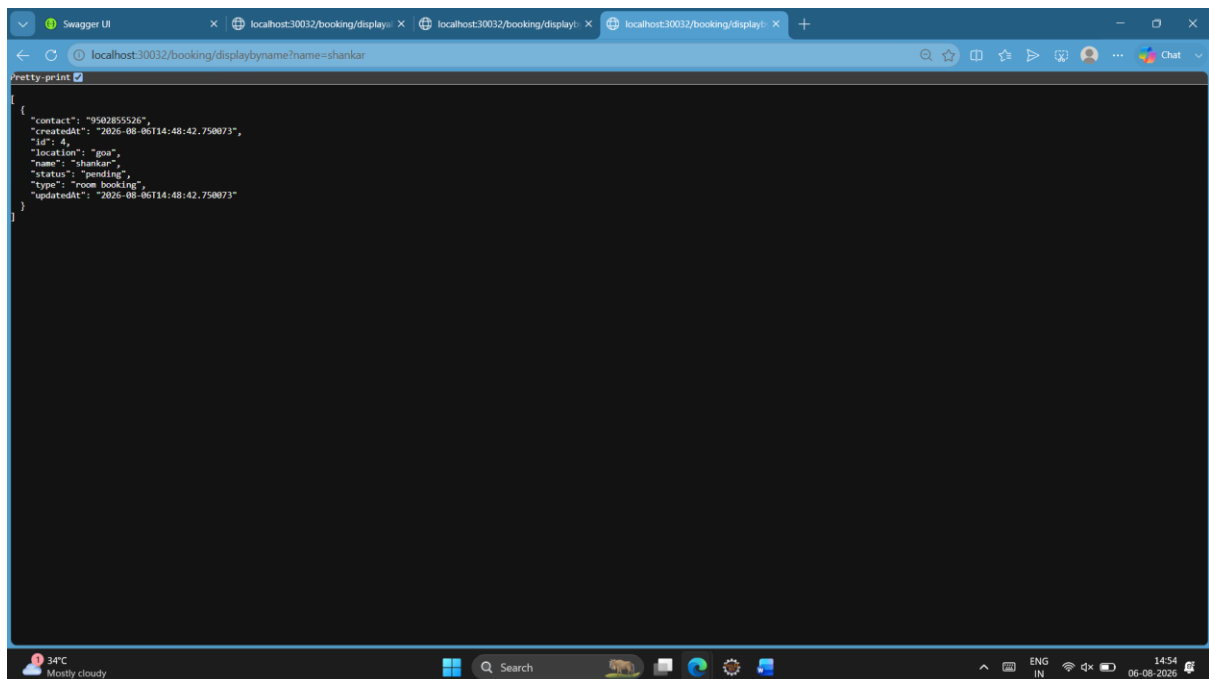
<http://localhost:30032/booking/displayallbookings>



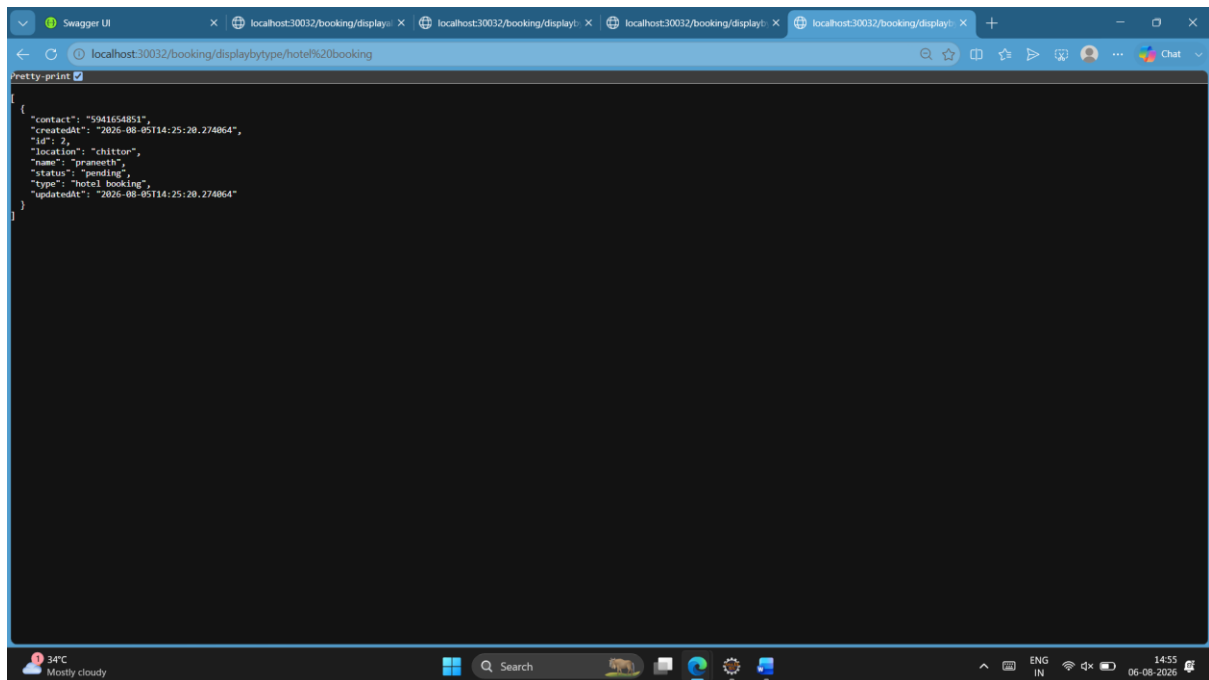
<http://localhost:30032/booking/displaybyid?id=2>



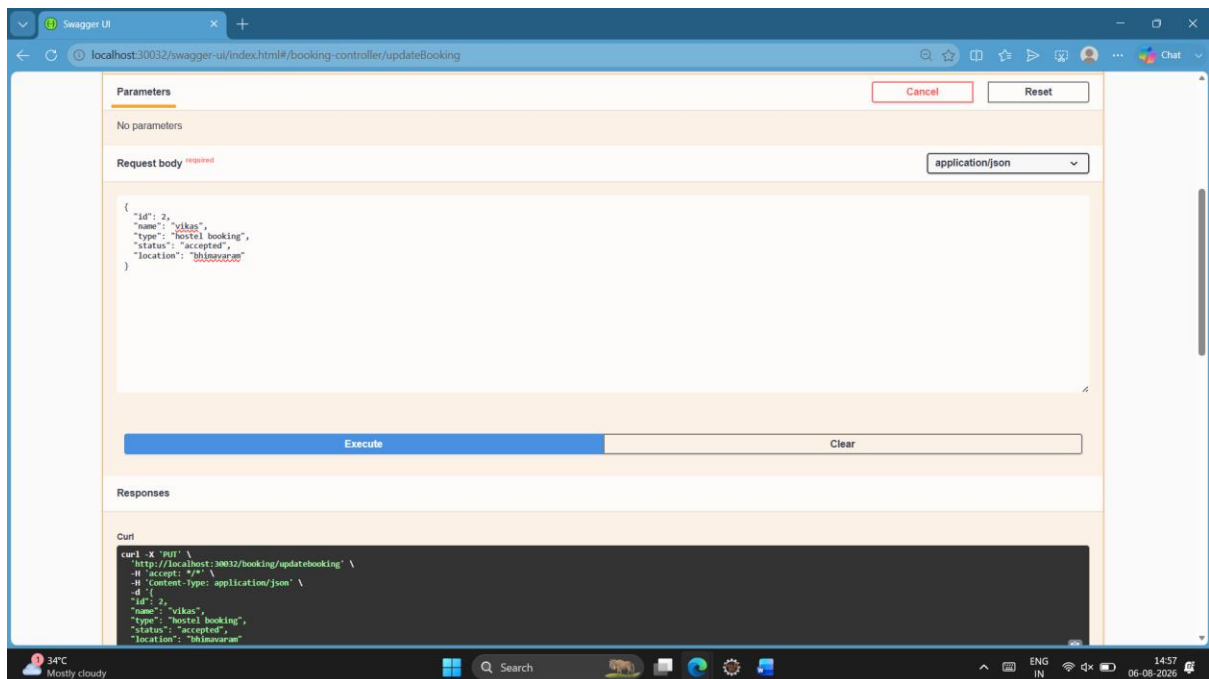
<http://localhost:30032/booking/displaybyname?name=shankar>



<http://localhost:30032/booking/displaybytype/hotel%20booking>



<http://localhost:30032/booking/updatebooking>



The image shows the Swagger UI interface for a REST API. The selected endpoint is `PUT /booking/updatebooking`. The response details are as follows:

- Request URL:** `http://localhost:30032/booking/updatebooking`
- Server response:**
 - Code:** 200
 - Response body:**

```
{  "contact": "991054853",  "createddt": "2026-08-05T14:25:20.274064",  "id": 2,  "location": "Bhinavaram",  "name": "vikas",  "status": "accepted",  "type": "hotel booking",  "updateddt": "2026-08-06T14:36:48.833305"}
```
 - Response headers:**

```
connection: keep-alivecontent-type: application/jsondate: Thu, 06 Aug 2026 09:26:48 GMTkeep-alive: timeout=60transfer-encoding: chunked
```
- Responses:**

Code	Description	Links
200	OK	No links

<http://localhost:30032/booking/deletebooking/2>

The image shows the Swagger UI interface for a REST API. The selected endpoint is `DELETE /booking/deletebooking/{id}`. The response details are as follows:

- Parameters:**

Name	Description
<code>id</code> * required	Integer(\$int64) (path)
- Responses:**

Curl

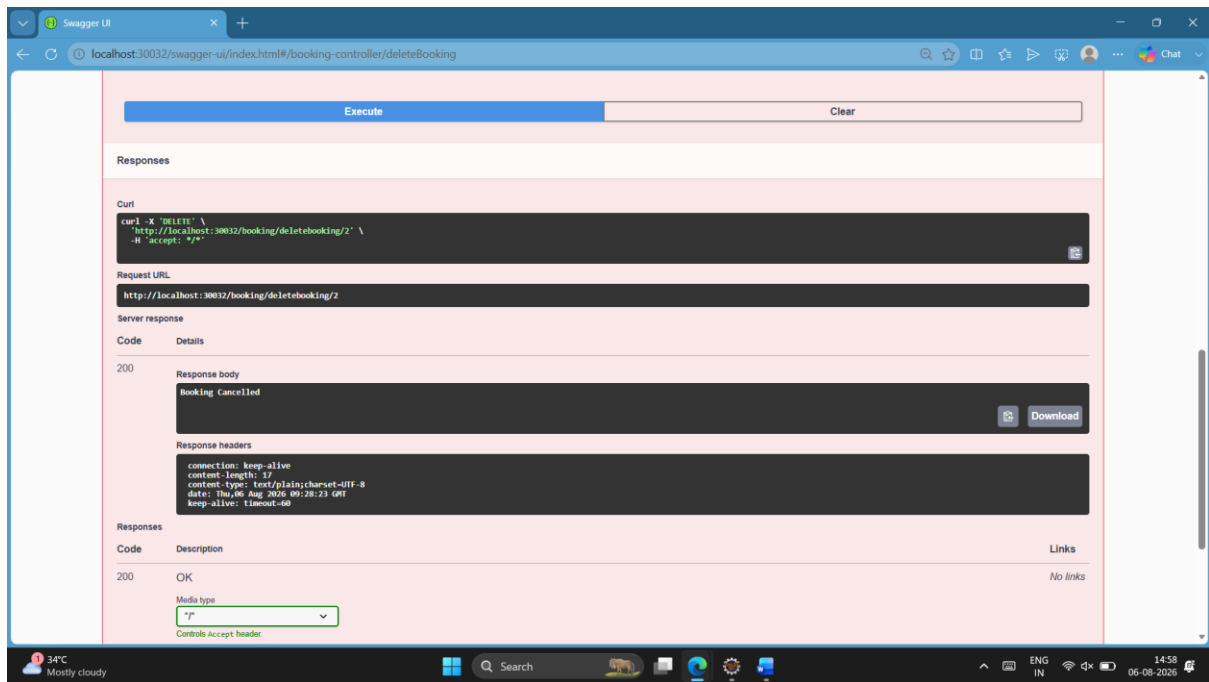
```
curl -X 'DELETE' \  'http://localhost:30032/booking/deletebooking/6' \  -H 'accept: */*'
```

Request URL

```
http://localhost:30032/booking/deletebooking/6
```

Server response

Code Details



DataBase:

